Research Brief: Django + Wagtail REST Framework (Ninja) and SSR/SSG, key considerations and implementations in backend

1. Problem Decomposition (chain-of-thought, 3–5 sub-problems)

At the HTTP/OS level this integration requires the existing modular Django Ninja setup (`api/v1/blog.py` , `api/v1/portfolio.py` , `api/v1/calculators.py`) to expose typed JSON endpoints that Next.js 15 App Router Server Components can consume via `fetch` for Portfolio (SSG/ISR), Blog (SSR/ISR + RSS), and Calc Hub (CSR). Wagtail Page models (located in `apps/blog/models.py` and `apps/portfolio/models.py`) must be serialized with `.specific()` dispatch, rich-text rendered via `expand_db_html` , and image renditions optimised for S3/R2 + `next/image` . Publish signals (already present in `apps/blog/signals.py`) must POST to Vercel's `/api/revalidate` (tag-based ISR). Draft/preview content must be isolated via tokens. Internally: Ninja (Pydantic v2) validates/serialises; Wagtail does polymorphic dispatch + signals; Django ORM + Redis cache (in `config/settings/`) + Gunicorn (Render/DO) serve low-latency responses; CORS must be Vercel-only. What can go wrong: N+1 queries on list endpoints → high TTFB hurting LCP/SEO and Vercel build times; draft leakage into public ISR cache; CORS preflight failures; stale content after publish; unoptimised media payloads inflating payload size. Each library's role: Wagtail = content tree + signals; Ninja = OpenAPI-first API; Redis = cache layer.

2. Top 5 Research Findings (2025–2026)

Finding 1 (IMPACT: productivity)

Official Wagtail + Django Ninja pattern using `ModelSchema` + resolver methods for polymorphic Wagtail Pages, `content_type` dispatch, and rich-text expansion. Fits directly into the existing modular `api/v1/` structure (one router per file).

Source + version: Wagtail 7.3+ official Ninja guide + Django Ninja 1.6+ (Pydantic 2).

Code snippet:

```
Python
```

```
# api/v1/blog.py
from ninja import Router
from ninja import Field, ModelSchema
from wagtail.models import Page
from wagtail.rich_text import expand_db_html
from apps.blog.models import BlogPage

router = Router(tags=["blog"])

class BasePageSchema(ModelSchema):
    url: str = Field(None, alias="get_url")
    content_type: str
    @staticmethod
    def resolve_content_type(page: Page) -> str:
        return page.specific_class._meta.model_name
    class Config:
        model = Page
        model_fields = ["id", "title", "slug", "last_published_at"]

class BlogPageSchema(BasePageSchema, ModelSchema):
    content_type: str = "blogpage"
    body: str
    @staticmethod
    def resolve_body(page: BlogPage) -> str:
        return expand_db_html(page.body)
    class Config(BasePageSchema.Config):
        model = BlogPage
        model_fields = ["intro"]
```

Finding 2 (IMPACT: performance)

`.live().public().specific().prefetch_related()` + Redis caching (Django cache backend) on list/detail endpoints inside existing `api/v1/` files. Guarantees TTFB <50 ms for Next.js ISR/SSG fetches and prevents DB load spikes (LCP/SEO win, lower Vercel build latency).

Source + version: Wagtail 7.3+ ORM best practices + Django 5.1 cache framework (Redis) + Django Ninja 1.6+.

⚠️ DEPRECATED: Pre-2024 raw `Page.objects.all()` without filters (leaks drafts, N+1 queries).

Code snippet:

```python
# api/v1/portfolio.py
from django.core.cache import cache

@router.get("/pages/")
def list_pages(request):
    key = "public_portfolio_pages"
    pages = cache.get(key)
    if pages is None:
        qs =
Page.objects.live().public().exclude(id=1).specific().prefetch_related("authors")
        pages = list(qs)
        cache.set(key, pages, timeout=300)
    return pages
```

Finding 3 (IMPACT: performance + seo)

Image renditions resolved via `get_renditions` (WebP + django-storages S3/R2) + rich-text expansion directly in Ninja schemas. Delivers `next/image`-ready URLs and SEO-ready HTML (zero client processing, reduced bundle size).

Source + version: Wagtail 7.3+ Image API + Ninja resolvers + django-storages 1.14+.

Code snippet:

```python
from wagtail.images.models import AbstractRendition
```

```
class RenditionSchema(ModelSchema):
    url: str = Field(None, alias="file.url")
    class Config:
        model = AbstractRendition
        model_fields = ["width", "height"]

# Inside BlogPageSchema
    main_image: list[RenditionSchema] = []
    @staticmethod
    def resolve_main_image(page):
        if image := getattr(page, "main_image", None):
            return list(image.get_renditions("fill-1200x630|format-
webp").values())
        return []
```

Finding 4 (IMPACT: seo)

Enhance existing `apps/blog/signals.py` with `page_published` / `page_unpublished` signals → POST to Next.js tag-based ISR revalidation. Keeps content fresh without full Vercel rebuilds (zero stale pages).

Source + version: Wagtail 7.3+ signals + Next.js 15 App Router `revalidateTag` .

Code snippet:

```python
# apps/blog/signals.py
from django.dispatch import receiver
from wagtail.signals import page_published, page_unpublished
import requests, os

@receiver([page_published, page_unpublished])
def trigger_isr_revalidate(sender, instance, **kwargs):
    requests.post(
        f"https://{os.getenv('FRONTEND_DOMAIN')}/api/revalidate",
        json={"tag": f"page-{instance.id}"},
        headers={"Authorization": f"Bearer
{os.getenv('NEXT_REVALIDATE_SECRET')}"},
        timeout=5
    )
```

Finding 5 (IMPACT: security)

`wagtail-headless-preview` + `django-cors-headers` (Vercel origin only) + token-based preview endpoint (add to `api/v1/auth.py` or new preview router). Isolates drafts from public API/ISR cache while enabling secure Next.js Draft Mode.

Source + version: Wagtail 7.3+ headless preview docs + wagtail-headless-preview 0.8+ + django-cors-headers 4.6+.

Code snippet:

```python
# config/settings/production.py
CORS_ALLOWED_ORIGINS = [f"https://{os.getenv('FRONTEND_DOMAIN')}"]
WAGTAIL_HEADLESS_PREVIEW = {"client_root_url":
f"https://{os.getenv('FRONTEND_DOMAIN')}"}

# api/v1/auth.py (or dedicated router)
from wagtail_headless_preview.models import PagePreview
from django.shortcuts import get_object_or_404
@router.get("/preview/{token}/")
def preview_page(request, token: str):
    preview = get_object_or_404(PagePreview, token=token)
    return preview.page.specific
```

3. Warnings & Corrections

- **Common mistake**: Raw `Page.objects` queries in `api/v1/` files → draft leaks + N+1 queries + slow ISR.
  Correction: Always `.live().public().specific().prefetch_related(...)` + Redis cache (Finding 2).
- **Common mistake**: Plain RichText/StreamField fields → internal JSON instead of rendered HTML.
  Correction: Add `resolve_*` methods with `expand_db_html` (Finding 3).
- **Common mistake**: No publish revalidation using existing `signals.py` → stale ISR content.
  Correction: Add signal handler (Finding 4).

- **Common mistake**: Broad or missing CORS → cross-origin fetch failures from Vercel.
  Correction: Restrict `CORS_ALLOWED_ORIGINS` to Vercel domain + `wagtail-headless-preview` (Finding 5).
- **Common mistake**: No media optimisation from S3/R2.
  Correction: `get_renditions` with WebP in schemas (Finding 3).

4. Compatibility Matrix

Library	Min Version	Tested With	Notes
Wagtail	6.0+	7.3+ (LTS)	Official Ninja + headless preview
Django	5.0+	5.1+	Full ORM/cache support
Django Ninja	1.x	1.6+	Pydantic 2, OpenAPI-first
wagtail-headless-preview	0.8+	0.8+	Draft/preview tokens
django-cors-headers	4.0+	4.6+	Vercel-only origins
django-storages	1.14+	1.14+	S3/R2 + renditions

5. Recommended Implementation Sequence

1. Add missing packages to `requirements.txt` (`django-ninja`, `wagtail-headless-preview`, `django-cors-headers[redis]`, `django-storages[boto3]` or google).
2. Update `config/settings/base.py` + `production.py`: CORS, Redis cache, storages, WAGTAIL_HEADLESS_PREVIEW.
3. Implement `BasePageSchema` + specific schemas + routers in `api/v1/blog.py` and `api/v1/portfolio.py` (Finding 1).
4. Add optimised list/detail endpoints with Redis caching (Finding 2).
5. Add image/rich-text resolvers (Finding 3).
6. Enhance `apps/blog/signals.py` for ISR revalidation (Finding 4) + connect routers in `config/urls.py`.
7. Add preview endpoint in `api/v1/auth` (Finding 5) + test with Next.js `draft`

7. Add preview endpoint in `api/v1/auth.py` (Finding 5) + test with Next.js `fetch`.

6. Directives for Next Step

- **What to pass to Prompt #3 (backend code generator)**: This full Research Brief + the provided file tree + any existing content from `apps/blog/models.py`, `apps/portfolio/models.py`, `config/settings/`, and current `api/v1/` files. Generate complete schemas/routers, signals updates, preview endpoint, and settings config.
- **What to pass to Prompt #4 (frontend code generator)**: This brief + the generated OpenAPI JSON from `/api/docs`. Generate Next.js 15 App Router Server Components using `fetch` with `revalidateTag`, ISR `/api/revalidate` route, and Draft Mode preview logic.
- **Any open questions the developer must resolve before coding**: (1) Exact Vercel domain and `NEXT_REVALIDATE_SECRET` for CORS/signals? (2) Which Wagtail Page models live in `apps/blog/` vs `apps/portfolio/`? (3) Preferred Redis connection string / timeout in production settings? (4) Any custom StreamField blocks requiring dedicated schema resolvers?